

Министерство науки и высшего образования Российской Федерации  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ  
ИТМО

ОТЧЕТ  
ПО ЛАБОРАТОРНОЙ РАБОТЕ №6  
«Работа с БД в СУБД MongoDB »  
по дисциплине «Проектирование и реализация баз данных»

Обучающийся Аксянова Алина Рустамовна  
Факультет прикладной информатики  
Группа К3240  
Направление подготовки 09.03.03 Прикладная информатика  
Образовательная программа Мобильные и сетевые технологии 2023  
Преподаватель Говорова Марина Михайловна

Санкт-Петербург  
2025/2026

## СОДЕРЖАНИЕ

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>Цель работы.....</b>                                                               | <b>4</b>  |
| <b>Практическое задание.....</b>                                                      | <b>5</b>  |
| <b>Выполнение.....</b>                                                                | <b>9</b>  |
| Лаб 6.1.....                                                                          | 9         |
| Лаб 6.2.....                                                                          | 13        |
| Задание 2.1.1 вставка документов в коллекцию.....                                     | 13        |
| Задание 2.2.1 выборка данных из бд.....                                               | 17        |
| Задание 2.2.2 исключить поля.....                                                     | 20        |
| Задание 2.2.3 обратный порядок добавления.....                                        | 21        |
| Задание 2.1.4 первое предпочтение без идентификатора.....                             | 22        |
| Задание 2.3.1 самки весом от 500 до 700 кг.....                                       | 22        |
| Задание 2.3.2 самцы весом от полутонны, любят grape и lemon, исключив вывод айди..... | 23        |
| Задание 2.3.3 единороги без ключа vampires.....                                       | 23        |
| Задание 2.3.4 имена самцов и первое предпочтение.....                                 | 23        |
| Задание 3.1.1 Запрос к вложенным объектам.....                                        | 24        |
| Задание 3.1.2 JavaScript и курсоры.....                                               | 25        |
| Задание 3.2.1 количество самок весом от 500 до 600 кг.....                            | 25        |
| Задание 3.2.2 список предпочтений.....                                                | 25        |
| Задание 3.2.3 количество единорогов обоих полов.....                                  | 26        |
| Задание 3.3.1 Редактирование данных, save.....                                        | 26        |
| Задание 3.3.2 изменить Ayna и проверить.....                                          | 27        |
| Задание 3.3.3 изменить Raleigh и проверить.....                                       | 27        |
| Задание 3.3.4 всем самцам увеличить количество убитых вампиров на 5 и проверить.....  | 28        |
| Задание 3.3.5 изменить: мэра Portland теперь беспартийный и проверить.....            | 28        |
| Задание 3.3.6 изменить Pilot , любит chocolate, проверить.....                        | 29        |
| Задание 3.3.7 изменить Aurora, любит sugar и lemon, проверить.....                    | 29        |
| Задание 3.4.1. Удаление документов из коллекции towns.....                            | 30        |
| Задание 4.1.1. Ссылки в бд.....                                                       | 31        |
| Задание 4.2.1. Настройка индексов.....                                                | 32        |
| Задание 4.3.1. Управление индексами.....                                              | 32        |
| Задание 4.4.1. План запроса.....                                                      | 33        |
| <b>Вывод.....</b>                                                                     | <b>35</b> |



## **Цель работы**

- 1) овладеть практическими навыками установки СУБД MongoDB.
- 2) овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

## Практическое задание

### 6.1 ВВЕДЕНИЕ В СУБД MONGODB. УСТАНОВКА MONGODB. НАЧАЛО РАБОТЫ С БД

1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
  - a) db.help()
  - b) db.help
  - c) db.stats()
4. Создайте БД learn.
5. Получите список доступных БД.
6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
7. Просмотрите список текущих коллекций.
8. Переименуйте коллекцию unicorns.
9. Просмотрите статистику коллекции.
10. Удалите коллекцию.
11. Удалите БД learn.

6.2

#### **Практическое задание 2.1.1:**

- 1) *Создайте базу данных learn.*
- 2) *Заполните коллекцию единорогов unicorns:*
- 3) *Используя второй способ, вставьте в коллекцию единорогов документ:*
- 4) *Проверьте содержимое коллекции с помощью метода find.*

#### **Практическое задание 2.2.1:**

1) *Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.*

2) *Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.*

#### **Практическое задание 2.2.2:**

*Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.*

#### **Практическое задание 2.2.3:**

*Вывести список единорогов в обратном порядке добавления.*

#### **Практическое задание 2.1.4:**

*Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.*

#### **Практическое задание 2.3.1:**

*Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.*

**Практическое задание 2.3.2:**

*Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.*

**Практическое задание 2.3.3:**

*Найти всех единорогов, не имеющих ключ vampires.*

**Практическое задание 2.3.4:**

*Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.*

**Практическое задание 3.1.1:**

- 1) *Создайте коллекцию towns, включающую следующие документы:*
- 2) *Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.*
- 3) *Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.*

**Практическое задание 3.1.2:**

- 1) *Сформировать функцию для вывода списка самцов единорогов.*
- 2) *Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.*
- 3) *Вывести результат, используя forEach.*
- 4) *Содержание коллекции единорогов unicorns:*

**Практическое задание 3.2.1:**

*Вывести количество самок единорогов весом от полутонны до 600 кг.*

**Практическое задание 3.2.2:**

*Вывести список предпочтений.*

**Практическое задание 3.2.3:**

*Посчитать количество особей единорогов обоих полов.*

**Практическое задание 3.3.1:**

1. *Выполнить команду:*  

```
> db.unicorns.save({name: 'Barney', loves: ['grape'],  
weight: 340, gender: 'm'})
```

2. *Проверить содержимое коллекции unicorns.*

**Практическое задание 3.3.2:**

1. Для самки единорога *Ayna* внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции *unicorns*.

#### **Практическое задание 3.3.3:**

1. Для самца единорога *Raleigh* внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции *unicorns*.

#### **Практическое задание 3.3.4:**

1. Всем самцам единорогов увеличить количество убитых вампиров на 5.
2. Проверить содержимое коллекции *unicorns*.

#### **Практическое задание 3.3.5:**

1. Изменить информацию о городе *Портланд*: мэр этого города теперь беспартийный.
2. Проверить содержимое коллекции *towns*.

#### **Практическое задание 3.3.6:**

1. Изменить информацию о самце единорога *Pilot*: теперь он любит и шоколад.
2. Проверить содержимое коллекции *unicorns*.

#### **Практическое задание 3.3.7:**

1. Изменить информацию о самке единорога *Aurora*: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции *unicorns*.

#### **Практическое задание 3.4.1:**

- 1) Создайте коллекцию *towns*, включающую следующие документы:
- 2) Удалите документы с беспартийными мэрами.
- 3) Проверьте содержание коллекции.
- 4) Очистите коллекцию.
- 5) Просмотрите список доступных коллекций.

#### **Практическое задание 4.1.1:**

- 1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.
- 2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
- 3) Проверьте содержание коллекции единорогов.
- 4) Содержание коллекции единорогов *unicorns*:

#### **Практическое задание 4.2.1:**

1. Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.
2. Содержание коллекции единорогов `unicorns`:

#### **Практическое задание 4.3.1:**

- 1) Получите информацию о всех индексах коллекции `unicorns`.
- 2) Удалите все индексы, кроме индекса для идентификатора.
- 3) Попробуйте удалить индекс для идентификатора.

#### **Практическое задание 4.4.1:**

- 1) Создайте объемную коллекцию `numbers`, задействовав курсор:  

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)
- 4) Создайте индекс для ключа `value`.
- 5) Получите информацию о всех индексах коллекции `numbers`.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?



# Выполнение

## Лаб 6.1

- 1) Установите MongoDB для обоих типов систем (32/64 бита). Проверьте работоспособность системы запуском клиента mongo.

```
mongosh mongodb://127.0.0.1 x + v
Please enter a MongoDB connection string (Default: mongodb://localhost/):

Current Mongosh Log ID: 6a1835c5e79b46ad95abc113
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.3
Using MongoDB:      8.3.2
Using Mongosh:       2.8.3

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-05-28T15:07:48.425+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> |
```

## 2) Выполните методы:

- db.help()

```
test> db.help()

Database Class:

getMongo           Returns the current database connection
getName            Returns the name of the DB
getCollectionNames Returns an array containing the names of all collections in the current database.
e.
getCollectionInfos Returns an array of documents with collection information, i.e. collection name
and options, for the current database.
runCommand          Runs an arbitrary command on the database.
adminCommand        Runs an arbitrary command against the admin database.
aggregate           Runs a specified admin/diagnostic pipeline which does not require an underlying
collection.
getSiblingDB        Returns another database without modifying the db variable in the shell environment.
getCollection        Returns a collection or a view object that is functionally equivalent to using
the db.<collectionName>.
dropDatabase         Removes the current database, deleting the associated data files.
createUser           Creates a new user for the database on which the method is run. db.createUser()
returns a duplicate user error if the user already exists on the database.
updateUser           Updates the user's profile on the database on which you run the method. An update to a field completely replaces the previous field's values. This includes updates to the user's roles array.
changeUserPassword   Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
logout              Ends the current authentication session. This function has no effect if the current session is not authenticated.
dropUser             Removes the user from the current database.
dropAllUsers         Removes all users from the current database.
auth                Allows a user to authenticate to the database from within the shell.
grantRolesToUser     Grants additional roles to a user.
revokeRolesFromUser  Removes a one or more roles from a user on the current database.
getUser             Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
```

нам вывело список доступных методов для работы с базой данных

## - db.help

```
test> db.help

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current database.
e.
  getCollectionInfos Returns an array of documents with collection information, i.e. collection name
  and options, for the current database.
  runCommand         Runs an arbitrary command on the database.
  adminCommand       Runs an arbitrary command against the admin database.
  aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying
  collection.
  getSiblingDB       Returns another database without modifying the db variable in the shell environment.
  getCollection      Returns a collection or a view object that is functionally equivalent to using
  the db.<collectionName>.
  dropDatabase       Removes the current database, deleting the associated data files.
  createUser         Creates a new user for the database on which the method is run. db.createUser()
  returns a duplicate user error if the user already exists on the database.
  updateUser         Updates the user's profile on the database on which you run the method. An update
  to a field completely replaces the previous field's values. This includes updates to the user's roles array.
  changeUserPassword Updates a user's password. Run the method in the database where the user is defined, i.e. the database you created the user.
  logout             Ends the current authentication session. This function has no effect if the current session is not authenticated.
  dropUser           Removes the user from the current database.
  dropAllUsers       Removes all users from the current database.
  auth              Allows a user to authenticate to the database from within the shell.
  grantRolesToUser   Grants additional roles to a user.
  revokeRolesFromUser Removes a one or more roles from a user on the current database.
  getUser            Returns user information for a specified user. Run this method on the user's database. The user must exist on the database on which the method runs.
```

## - db.stats()

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

Команда показывает статистику текущей базы данных: размер, количество коллекций, количество документов, количество индексов.

2) создаю бд learn и смотрю список всех существующих бд:

```
test> use learn
switched to db learn
learn> show dbs
admin    40.00 KiB
config   60.00 KiB
local    40.00 KiB
learn> |
```

3) создаю коллекцию unicorns

```
learn> db.unicorns.insertOne({
|   name: 'Aurora',
|   gender: 'f',
|   weight: 450
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a183799e79b46ad95abc114')
}
learn>
```

4) смотрю список коллекций

```
learn> show collections
unicorns
```

5) переименовываю коллекцию

```
learn> db.unicorns.renameCollection("new_unicorns")
{ ok: 1 }
learn>
```

## 6) смотрю статистику коллекции

```
learn> db.new_unicorns.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=none,durable_timestamp=none,read_timestamp=none,write_timestamp=off),block_allocation=best,block_compressor=snappy,block_manager=default,cache_resident=false,checksum=on,colgroups=,collator=,columns=,dictionary=0,disaggregated=(page_log=,storage_tier=none),encryption=(keyid=,name=)exclusive=false,extractor=,format=btree,huffman_key=,huffman_value=,ignore_in_memory_cache_size=false,immutable=false,import=(compare_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,panic_corrupt=true,repair=false),in_memory=false,ingest=,internal_item_max=0,internal_key_max=0,internal_key_truncate=true,internal_page_max=4KB,key_format=q,key_gap=10,leaf_item_max=0,leaf_key_max=0,leaf_page_max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=,bloom=,bloom_bit_count=,bloom_config=,bloom_hash_count=,bloom_oldest=,chunk_count_limits=,chunk_max=,chunk_size=,merge_custom=(prefix=,start_generation=,suffix=),merge_max=,merge_min=),memory_page_image_max=0,memory_page_max=10M,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,prefix_compression_min=4,source=,split_deepen_min_child=0,split_deepen_per_child=0,split_pct=90,stable=,tiered_storage=(auth_token=,buckets=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=file,value_format=u,verbose=[],write_timestamp_usage=none',
    type: 'file',
    uri: 'statistics:table:collection-47e523e0-3c9d-4795-8021-524dc221cee7',
    version: 'WiredTiger 12.0.0: (November 15, 2024)',
    autoccommit: {
      'retries for readonly operations': 0,
      'retries for update operations': 0
    },
    backup: {
      'total modified incremental blocks with compressed data': 0,
      'total modified incremental blocks without compressed data': 0
    },
    'block-disagg': {
      'Bytes read from the shared history store in SLS': 0,
      'Bytes written to the shared history store in SLS': 0,
      'Disaggregated block manager get': 0,
      'rollback to stable restored tombstones from history store': 0,
      'rollback to stable restored updates from history store': 0,
      'rollback to stable skipped btree's': 0,
      'rollback to stable skipping delete rle': 0,
      'rollback to stable skipping stable rle': 0,
      'rollback to stable sweeping history store keys': 0,
      'rollback to stable tombstones from history store that would have been restored in non-dryrun mode': 0,
      'rollback to stable updates from history store that would have been restored in non-dryrun mode': 0,
      'rollback to stable updates removed from history store': 0,
      'rollback to stable updates that would have been removed from history store in non-dryrun mode': 0,
      'update conflicts': 0
    }
  },
  sharded: false,
  size: 65,
  count: 1,
  numOrphanDocs: 0,
  storageSize: 20480,
  totalIndexSize: 20480,
  totalSize: 40960,
  indexSizes: { _id: 20480 },
  avgObjSize: 65,
  ns: 'learn.new_unicorns',
  nindexes: 1,
  scaleFactor: 1
}
learn> |
```

## 7) удаляю коллекцию

```
learn> db.new_unicorns.drop()
true
learn>
```

## 8) удаляю бд

```
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn> |
```

## Лаб 6.2

### Задание 2.1.1 вставка документов в коллекцию

Создаю бд learn и заполняю коллекцию единорогов unicorns

```
test> use learn
switched to db learn
learn> db.unicorns.insertMany([
  {name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63},
  {name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43},
  {name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182},
  {name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99},
  {name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80},
  {name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40},
  {name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39},
  {name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2},
  {name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33},
  {name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54},
  {name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a183bc97b645a425cabcl14'),
    '1': ObjectId('6a183bc97b645a425cabcl15'),
    '2': ObjectId('6a183bc97b645a425cabcl16'),
    '3': ObjectId('6a183bc97b645a425cabcl17'),
    '4': ObjectId('6a183bc97b645a425cabcl18'),
    '5': ObjectId('6a183bc97b645a425cabcl19'),
    '6': ObjectId('6a183bc97b645a425cabcl1a'),
    '7': ObjectId('6a183bc97b645a425cabcl1b'),
    '8': ObjectId('6a183bc97b645a425cabcl1c'),
    '9': ObjectId('6a183bc97b645a425cabcl1d'),
    '10': ObjectId('6a183bc97b645a425cabcl1e')
  }
}
learn>
```

вторым способом вставки добавляю единорогов:

```
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
| db.unicorns.insertOne(document)
{
  acknowledged: true,
  insertedId: ObjectId('6a183c5b7b645a425cabcl1f')
}
```

проверяю содержимое коллекции

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a183bc97b645a425cabcl14'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl15'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl16'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl17'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl18'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl19'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl1a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl1b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl1c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl1d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a183bc97b645a425cabcl1e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f',
    vampires: 54
  },
  {
    _id: ObjectId('6a183c5b7b645a425cabcl1f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
]
```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a183bc97b645a425cab114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a183bc97b645a425cab115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a183bc97b645a425cab116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a183bc97b645a425cab117'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a183bc97b645a425cab118'),
```

```
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a183bc97b645a425cab119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
```

```
vampires: 33
},
{
  _id: ObjectId('6a183bc97b645a425cab11d'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a183bc97b645a425cab11e'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a183c5b7b645a425cab11f'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
]
learn>
```



## Задание 2.2.1 выборка данных из бд

Список самцов, отсортированный по имени:

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a183c5b7b645a425cab11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a183bc97b645a425cab114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
```

```
    _id: ObjectId('6a183bc97b645a425cab11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a183bc97b645a425cab117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a183bc97b645a425cab116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn> |
```

Список самок, первые 3, сортировка по имени:

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a183bc97b645a425cabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a183bc97b645a425cabc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a183bc97b645a425cabc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> |
```

Самки, которые любят carrot:

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a183bc97b645a425cabc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a183bc97b645a425cabc118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a183bc97b645a425cabc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

Ограничиваю первой особью через findOne:

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a183bc97b645a425cab115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn>
```

Через limit:

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a183bc97b645a425cab115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>
```

### Задание 2.2.2 исключить поля

список самцов единорогов, исключив из результата информацию о предпочтениях и поле.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {loves: 0, gender: 0}
| )
[
  {
    _id: ObjectId('6a183bc97b645a425cab114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a183bc97b645a425cab116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a183bc97b645a425cab117'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11b'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11d'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a183c5b7b645a425cab11f'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
```

### Задание 2.2.3 обратный порядок добавления

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a183c5b7b645a425cab11f'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a183bc97b645a425cab11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a183bc97b645a425cab119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a183bc97b645a425cab118'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a183bc97b645a425cab117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a183bc97b645a425cab116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a183bc97b645a425cab115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a183bc97b645a425cab114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  }
]
```

```
{
  _id: ObjectId('6a183bc97b645a425cab115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a183bc97b645a425cab114'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
},
{
  _id: ObjectId('6a183bc97b645a425cab116'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a183bc97b645a425cab117'),
  name: 'Rooooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
{
  _id: ObjectId('6a183bc97b645a425cab118'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a183bc97b645a425cab119'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  _id: ObjectId('6a183bc97b645a425cab11a'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a183bc97b645a425cab11b'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar' ],
  weight: 421,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a183bc97b645a425cab11c'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a183bc97b645a425cab11d'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a183bc97b645a425cab11e'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a183c5b7b645a425cab11f'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
```

```
{
  _id: ObjectId('6a183bc97b645a425cab115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a183bc97b645a425cab114'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
}
```

### Задание 2.1.4 первое предпочтение без идентификатора

```
learn> db.unicorns.find(
|   {},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| )
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn> |
```

### Задание 2.3.1 самки весом от 500 до 700 кг

```
learn> db.unicorns.find(
|   {gender: 'f', weight: {$gte: 500, $lte: 700}},
|   {_id: 0}
| )
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

### Задание 2.3.2 самцы весом от полутонны, любят grape и lemon, ИСКЛЮЧИВ ВЫВОД айди

```
learn> db.unicorns.find(
|   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
|   {_id: 0}
| )
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> |
```

### Задание 2.3.3 единороги без ключа vampires

```
learn> db.unicorns.find(
|   {vampires: {$exists: false}}
| )
[
  {
    _id: ObjectId('6a183bc97b645a425cab11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

### Задание 2.3.4 имена самцов и первое предпочтение

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Roooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn>
```

### Задание 3.1.1 Запрос к вложенным объектам

создаю коллекцию towns

```
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a1841827b645a425cab120'),
    '1': ObjectId('6a1841827b645a425cab121'),
    '2': ObjectId('6a1841827b645a425cab122')
  }
}
learn>
```

список городов с независимыми мэрами

```
learn> db.towns.find(
  { "mayor.party": "I" },
  { _id: 0, name: 1, mayor: 1 }
)
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> |
```

список городов с беспартийными мэрами

```
learn> db.towns.find(
  { "mayor.party": { $exists: false } },
  { _id: 0, name: 1, mayor: 1 }
)
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```



### Задание 3.1.2 JavaScript и курсоры

Функция для вывода самцов единорогов

```
learn> fn = function() {  
|   return this.gender == "m";  
| }  
[Function: fn]  
learn> |
```

Курсор для первых двух самцов с сортировкой по имени

```
learn> var cursor = db.unicorns.find(fn).sort({name: 1}).limit(2);
```

Но при выполнении задания в новой версии mongosh возникла ошибка, так как метод find() больше не принимает JavaScript-функцию напрямую в качестве фильтра. Поэтому запрос был выполнен через оператор \$where

```
learn> cursor.forEach(function(obj) {  
|   print(obj.name);  
| })  
MongoInvalidArgumentError: Query filter must be a plain object or ObjectId  
learn>
```

```
learn> var cursor = db.unicorns.find({  
|   $where: function() {  
|     return this.gender == "m";  
|   }  
| }).sort({name: 1}).limit(2);
```

Вывод через forEach

```
learn> cursor.forEach(function(obj) {  
|   print(obj.name);  
| })  
Dunx  
Horny
```

### Задание 3.2.1 количество самок весом от 500 до 600 кг

```
learn> db.unicorns.find({  
|   gender: "f",  
|   weight: {$gte: 500, $lte: 600}  
| }).count()  
2  
learn>
```

### Задание 3.2.2 список предпочтений

```
learn> db.unicorns.distinct("loves")  
[  
  'apple',      'carrot',  
  'chocolate', 'energon',  
  'grape',      'lemon',  
  'papaya',     'redbull',  
  'strawberry', 'sugar',  
  'watermelon'  
]
```

### Задание 3.2.3 количество единорогов обоих полов

```
learn> db.unicorns.aggregate([
|   {
|     $group: {
|       _id: "$gender",
|       count: {$sum: 1}
|     }
|   }
| ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

### Задание 3.3.1 Редактирование данных, save

в новой версии нет save, заменяю на insertOne

```
learn> db.unicorns.save({
|   name: 'Barney',
|   loves: ['grape'],
|   weight: 340,
|   gender: 'm'
| })
TypeError: db.unicorns.save is not a function
learn> db.unicorns.insertOne({
|   name: 'Barney',
|   loves: ['grape'],
|   weight: 340,
|   gender: 'm'
| })
{
  acknowledged: true,
  insertedId: ObjectId('6a1848c57b645a425cab123')
}
```

проверяю содержимое

```
learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a1848c57b645a425cab123'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

### Задание 3.3.2 изменить Айна и проверить

```
learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a183bc97b645a425cab119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

### Задание 3.3.3 изменить Raleigh и проверить

```
learn> db.unicorns.updateOne(
|   {name: 'Raleigh'},
|   {$set: {loves: ['redbull']}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a183bc97b645a425cab11b'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

### Задание 3.3.4 всем самцам увеличить количество убитых вапмиров на 5 и проверить

```
learn> db.unicorns.updateMany(
|   {gender: 'm'},
|   {$inc: {vampires: 5}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: 'm'}, {_id: 0, name: 1, vampires: 1})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barney', vampires: 5 }
]
```

### Задание 3.3.5 изменить: мэр Portland теперь беспартийный и проверить

```
learn> db.towns.updateOne(
|   {name: 'Portland'},
|   {$unset: {"mayor.party": 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a1841827b645a425cab122'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

### Задание 3.3.6 изменить Pilot, любит chocolate, проверить

```
learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a183bc97b645a425cab11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

### Задание 3.3.7 изменить Aurora, любит sugar и lemon, проверить

```
learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a183bc97b645a425cab115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

### Задание 3.4.1. Удаление документов из коллекции towns

очищаю старую коллекцию и создаю новую

```
learn> db.towns.drop()
true
learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["statue of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a184b657b645a425cabc124'),
    '1': ObjectId('6a184b657b645a425cabc125'),
    '2': ObjectId('6a184b657b645a425cabc126')
  }
}
```

Удаляю документы с беспартийными мэрами и проверяю содержимое коллекции

```
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a184b657b645a425cabc125'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a184b657b645a425cabc126'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn>
```

очищаю коллекцию и проверяю доступные

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
learn> show collections
towns
unicorns
learn> db.towns.find()
```

### Задание 4.1.1. Ссылки в бд

Создаю коллекцию зон обитания

```
learn> db.habitats.insertMany([
  {
    _id: "forest",
    full_name: "Волшебный лес",
    description: "Зона обитания единорогов с большим количеством растений"
  },
  {
    _id: "mountain",
    full_name: "Горная долина",
    description: "Зона обитания сильных и крупных единорогов"
  },
  {
    _id: "lake",
    full_name: "Озеро света",
    description: "Зона обитания спокойных единорогов"
  }
])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountain', '2': 'lake' }
}
learn> |
```

Добавляю ссылки на зоны нескольким единорогам

```
learn> db.unicorns.updateOne(
  {
    name: "Aurora",
    $set: {habitat: {$ref: "habitats", $id: "lake"}}
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {
    name: "Horny",
    $set: {habitat: {$ref: "habitats", $id: "forest"}}
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
  {
    name: "Unicrom",
    $set: {habitat: {$ref: "habitats", $id: "mountain"}}
  }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Проверяю

```
learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {_id: 0, name: 1, habitat: 1}
| )
|
| [
|   { name: 'Horny', habitat: DBRef('habitats', 'forest') },
|   { name: 'Aurora', habitat: DBRef('habitats', 'lake') },
|   { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') }
| ]
learn> db.habitats.findOne({
|   _id: db.unicorns.findOne({name: "Aurora"}).habitat.oid
| })
|
| {
|   _id: 'lake',
|   full_name: 'Озеро света',
|   description: 'Зона обитания спокойных единорогов'
| }
learn>
```

### Задание 4.2.1. Настройка индексов

Создала индекс по полю name. Попробовала добавить документ с уже существующим именем Ауруса. выдало ошибку duplicate key error, что подтверждает работу ограничения уникальности.

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.insertOne({
|   name: "Aurora",
|   loves: ["apple"],
|   weight: 500,
|   gender: "f"
| })
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Aurora" }
learn>
```

### Задание 4.3.1. Управление индексами

Получаю информацию об индексах, удаляю все индексы, кроме \_id, проверяю индексы, пытаюсь удалить индекс \_id, но это невозможно, выдает ошибку

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```



### Задание 4.4.1. План запроса

Создаю объемную коллекцию numbers

```
learn> db.numbers.drop()
true
```

```
learn> for(i = 0; i < 100000; i++){
|   db.numbers.insertOne({value: i})
| }
{
  acknowledged: true,
  insertedId: ObjectId('6a1850b07b645a425cadb360')
}
```

выбираю последние 4 документа

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1850b07b645a425cadb360'), value: 99999 },
  { _id: ObjectId('6a1850b07b645a425cadb35f'), value: 99998 },
  { _id: ObjectId('6a1850b07b645a425cadb35e'), value: 99997 },
  { _id: ObjectId('6a1850b07b645a425cadb35d'), value: 99996 }
]
```

анализирую план выполнения запроса, executionTimeMillis = 160

```
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 160,
  totalKeysExamined: 0,
  totalDocsExamined: 127545,
  executionStages: {
    isCached: false,
    stage: 'SORT',
  }
}
```

Создаю индекс для ключа value и Получаю информацию о всех индексах коллекции numbers

```
learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

```
learn> |
type: 'simple',
inputStage: { stage: 'COLLSCAN', nss: 'learn.numbers', direction: 'forward' }
},
```

выполняю 2 запрос и анализирую, executionTimeMillis = 10

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a1850b07b645a425cadb360'), value: 99999 },
  { _id: ObjectId('6a1850b07b645a425cadb35f'), value: 99998 },
  { _id: ObjectId('6a1850b07b645a425cadb35e'), value: 99997 },
  { _id: ObjectId('6a1850b07b645a425cadb35d'), value: 99996 }
]
learn> db.numbers.find().sort({value: -1}).limit(4).explain("executionStats")
{
```

```
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 10,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
    executionStages: {
```

```
      winningPlan: {
        isCached: false,
        stage: 'LIMIT',
        limitAmount: 4,
        inputStage: {
          stage: 'FETCH',
          nss: 'learn.numbers',
          inputStage: {
            stage: 'IXSCAN',
```

До создания индекса запрос выполнялся с полным просмотром коллекции COLLSCAN. После создания индекса в плане выполнения появился IXSCAN. Запрос с индексом является более эффективным, так как MongoDB использует индекс и не просматривает всю коллекцию полностью.

## **Вывод**

В ходе лабораторной работы я познакомилась с базой данных MongoDB и научилась работать с консолью `mongosh`. Были выполнены операции добавления, поиска, изменения и удаления документов в коллекциях. Также была изучена работа с вложенными объектами, ссылками между документами и курсорами.

Кроме этого, были созданы индексы и выполнен анализ запросов с помощью `explain()`. В результате было видно, что запросы с индексом выполняются быстрее, чем без него, так как MongoDB использует индекс вместо полного перебора коллекции.